

LevelDB-分组Topics

Topics:

1. 存储（数据结构与写入）

1.1. WAL

需要回答的问题：

1. 为什么要有WAL？其作用是什么？
2. WriteBatch::Put函数的实现。数据（键值对）如何编码？
3. DBImpl::Write函数的实现。多线程并发写WAL的过程是什么？日志的Batch编码格式。
4. leveldb::log::Writer::AddRecord函数的实现。分为4种RecordType的作用是什么？为什么要分块？
5. 写入日志的数据什么时候被持久化（落盘）？

1.2. Memtable

需要回答的问题：

1. 为什么要有memtable？其作用是什么？
2. MakeRoomForWrite函数的作用？如何把旧Memtable冻结？写暂停是如何产生的？
3. WriteBatchInternal::InsertInto函数的实现。
4. SkipList是如何实现的？如何把数据插入SkipList？如何读取？
5. 如果一个键出现在多个 memtable 中或多个键出现在同一个memtable中，应向用户返回哪个？
6. 在冻结 memtable 之后，是否可能有一些线程仍然持有旧的 LSM 状态并写入这些不可变的 memtable？如何防止这种情况发生？
7. 当不可变的 memtable落盘后还有线程在读这个memtable怎么办？

1.3. 构建SSTable(合并触发)

需要回答的问题:

1. SSTable的结构是什么? 这么设计的好处是什么?
2. BlockBuilder的作用以及BlockBuilder::Add函数的实现。
3. TableBuilder的构造函数和TableBuilder::Rep结构体内字段的作用。
4. TableBuilder::Add函数的实现。
5. TableBuilder::Flush函数的实现。调用的时机是什么?
6. TableBuilder::WriteBlock函数的实现。
7. TableBuilder::WriteRawBlock函数的实现。
8. TableBuilder::Finish函数的实现。

2. 数据访问 (如何读数据)

2.1. 读取键值对

需要回答的问题:

1. 查找一个键值对的读流程是什么? 时间复杂度是多少? 为什么LSM-tree的读性能会弱于B+Tree?
2. DBImpl::Get函数的实现。快照的作用是什么?
3. Version::Get函数的实现。
4. Version::ForEachOverlapping函数的实现。
5. TableCache::Get函数的实现。
6. TableCache::FindTable函数的实现。BloomFilter是什么时候被加载到内存的?
7. Table::InternalGet函数的实现。BloomFilter是如何发挥作用的? 哪一步定位到目标Block的
8. Block::Iter::Seek函数的实现。

2.2. Cache

需要回答的问题:

1. Cache的作用是什么? 其缓存的粒度是什么? 缓存了哪些信息?

2. ShardedLRUCache类的实现。为什么要分片？
3. LRUCache类的实现。LRU的原理是什么？
4. LRUHandle类的实现。Handle结构体的作用是什么？

2.3. 扫描/Merge Iterator <<https://skyzh.github.io/mini-lsm/week1-02-merge-iterator.html#merge-iterator>>

需要回答的问题：

1. 为什么要有Merge Iterator <<https://skyzh.github.io/mini-lsm/week1-02-merge-iterator.html#merge-iterator>> ?
2. DBImpl::NewInternalIterator函数的实现。内部包含了哪些迭代器？MergingIterator是如何构造的？
3. 用户调用Iterator::SeekToFirst()时具体调用了哪些迭代器？
4. DBIter::SeekToFirst与DBIter::FindNextUserEntry函数的实现。
5. MergingIterator::SeekToFirst()与MergingIterator::FindSmallest函数的实现。
6. MergingIterator::Next()函数的实现。
7. 如何保证被删除的Key不会返回给用户？
8. 在迭代过程中有新的键值对插入，用户能否看到？

2.4. Snapshot

XXX

3. 数据管理（Manifest/创建/恢复数据库）

3.1. Manifest

需要回答的问题：

1. Manifest的作用是什么？存了什么信息？什么时候会更新Manifest？VersionSet的作用？
2. VersionSet::LogAndApply函数的实现。Manifest内数据存储的格式是什么？

3. VersionSet::Builder::Apply函数的实现。
4. VersionSet::WriteSnapshot函数的实现。为什么要创建快照？
5. VersionSet::Append函数的实现。VersionVersionSet、Version和VersionEdit的关系是什么？
6. 如何在启动时知道最新的Manifest是什么？

3.2. 创建数据库

需要回答的问题：

1. 创建数据库的流程是什么？创建数据库时创建了哪些文件，都有什么作用？
2. DBImpl类内都有哪些成员？其作用是什么？
3. DB::Open函数的实现。
4. DBImpl::Recover函数的实现。
5. DBImpl::NewDB函数的实现。
6. VersionSet::Recover函数的实现。CURRENT的作用是什么？

3.3. 从WAL日志恢复Memtable

需要回答的问题：

1. 读取WAL的流程是什么？
2. DBImpl::RecoverLogFile函数的实现。
3. log::Reader::ReadRecord函数的实现。
4. log::Reader::ReadPhysicalRecord函数的实现。一次从磁盘读取的数据量为多少？

3.4. 内存管理

XXX

4. 数据合并 (Compaction)

4.1. 合并策略

需要回答的问题:

1. 合并用来解决什么问题? 什么时候需要做合并? 哪些数据应该被合并? 合并后数据放到哪里?
2. DBImpl::MaybeScheduleCompaction函数的实现。什么时候会调用这个函数? 如何发起合并的?
3. DBImpl::BackgroundCompaction函数的实现。
4. VersionSet::PickCompaction函数的实现。compaction_score_如何计算的? 选择哪些文件合并?
5. Version::GetOverlappingInputs函数的实现。
6. VersionSet::SetupOtherInputs函数的实现。其解决了什么问题?
7. Compaction::IsTrivialMove函数的实现。其作用是什么?

4.2. 小合并 (Flush)

需要回答的问题:

1. 小合并的作用是什么? 带来了哪些问题?
2. DBImpl::CompactMemTable函数的实现。
3. DBImpl::WriteLevel0Table函数的实现。刷入磁盘的数据会写到哪一层?
4. BuildTable函数的实现。Env::NewWritableFile是如何确定底层实现的文件接口的?

4.3. 大合并 (Major compaction)

要回答的问题:

1. DBImpl::DoCompactionWork函数的实现。如果大合并中间有小合并怎么办? 遇到过时数据的处理逻辑?
2. VersionSet::MakeInputIterator函数的实现。
3. DBImpl::OpenCompactionOutputFile函数的实现。
4. Compaction::ShouldStopBefore函数的实现。
5. DBImpl::FinishCompactionOutputFile函数的实现。
6. 统计了合并中的那些信息?
7. DBImpl::InstallCompactionResults函数的实现。
8. VersionSet::LogAndApply函数的实现。其作用是什么?

9. 如何避免读写冲突？当某个迭代器正在迭代某个sstable文件的内容，而后台的major compaction进程完成了合并动作，试图删除该sstable文件。系统如何应对这个问题？